

GOOGLE CLOUD PROFESSIONAL CLOUD ARCHITECT

Section 5.2 — Managing Service Accounts

GCP Associate Cloud Engineer

Study Notes Study Notes April 2026

Exam Relevance

This topic is part of **Section 5: Configuring access and security (~17.5 % of the exam)**. You must know how to create service accounts, use them in IAM policies with minimum permissions, assign them to resources, manage IAM of a service account, manage impersonation, and create short-lived credentials.

1. Service Account Fundamentals

What Is a Service Account?

A service account is a **special type of Google account** used by applications, VMs, and services to authenticate and authorize API calls. Unlike user accounts, service accounts:

- Are not associated with a human user
- Authenticate using **keys** or **metadata** (not passwords)
- Can be assigned IAM roles like any other member
- Have the format: `SA_NAME@PROJECT_ID.iam.gserviceaccount.com`

Service Account Types

Type	Created By	Example
User-managed	You (the developer/admin)	<code>my-app@project.iam.gserviceaccount.com</code>
Default	Google (when you enable APIs)	<code>PROJECT_NUMBER-compute@developer.gserviceaccount.com</code>
Google-managed	Google (internal service agents)	<code>service-PROJECT_NUMBER@compute-system.iam.gserviceaccount.com</code>

Default Service Accounts

Service	Default SA Format	Default Role
Compute Engine	<code>PROJECT_NUMBER-compute@developer.gserviceaccount.com</code>	<code>roles/editor</code> (too broad!)
App Engine	<code>PROJECT_ID@appspot.gserviceaccount.com</code>	<code>roles/editor</code>

Important: Default service accounts have `roles/editor` which is **too permissive**. Best practice is to create **dedicated service accounts with minimal permissions**.

2. Creating Service Accounts

```
# Create a service account
gcloud iam service-accounts create my-app-sa \
  --display-name="My Application Service Account" \
  --description="Used by my-app to access Cloud Storage and Pub/Sub"

# The service account email will be:
# my-app-sa@PROJECT_ID.iam.gserviceaccount.com

# List service accounts
gcloud iam service-accounts list

# Describe a service account
gcloud iam service-accounts describe my-app-sa@PROJECT_ID.iam.gserviceaccount.com

# Update display name
gcloud iam service-accounts update my-app-sa@PROJECT_ID.iam.gserviceaccount.com \
  --display-name="Updated Display Name"

# Disable a service account (temporarily revoke access)
gcloud iam service-accounts disable my-app-sa@PROJECT_ID.iam.gserviceaccount.com

# Enable a disabled service account
gcloud iam service-accounts enable my-app-sa@PROJECT_ID.iam.gserviceaccount.com

# Delete a service account
gcloud iam service-accounts delete my-app-sa@PROJECT_ID.iam.gserviceaccount.com

# Undelete a service account (within 30 days)
gcloud iam service-accounts undelete UNIQUE_ID
```

Service Account Limits

- Maximum **100 user-managed service accounts** per project
- Each SA has a globally unique email address
- SA names: 6-30 characters, lowercase, hyphens allowed

3. Using Service Accounts in IAM Policies with Minimum Permissions

Granting Roles to Service Accounts

Service accounts are treated as **members/principals** in IAM:

```
# Grant a service account specific permissions
gcloud projects add-iam-policy-binding PROJECT_ID \
  --member="serviceAccount:my-app-sa@PROJECT_ID.iam.gserviceaccount.com" \
  --role="roles/storage.objectViewer"

# Grant multiple roles for different services
gcloud projects add-iam-policy-binding PROJECT_ID \
  --member="serviceAccount:my-app-sa@PROJECT_ID.iam.gserviceaccount.com" \
  --role="roles/pubsub.subscriber"

# Grant at the resource level (more restrictive = better)
gcloud storage buckets add-iam-policy-binding gs://my-bucket \
  --member="serviceAccount:my-app-sa@PROJECT_ID.iam.gserviceaccount.com" \
  --role="roles/storage.objectViewer"
```

Principle of Least Privilege for Service Accounts

Instead of this (BAD):

```
# DON'T: Grant Editor role to a service account
gcloud projects add-iam-policy-binding PROJECT_ID \
  --member="serviceAccount:my-sa@PROJECT_ID.iam.gserviceaccount.com" \
  --role="roles/editor"
```

Do this (GOOD):

```
# DO: Grant only the specific permissions needed
gcloud projects add-iam-policy-binding PROJECT_ID \
  --member="serviceAccount:my-sa@PROJECT_ID.iam.gserviceaccount.com" \
  --role="roles/storage.objectViewer"

gcloud projects add-iam-policy-binding PROJECT_ID \
  --member="serviceAccount:my-sa@PROJECT_ID.iam.gserviceaccount.com" \
  --role="roles/pubsub.subscriber"
```

Best Practices

1. **Create dedicated service accounts** per application/service
2. **Never use the default service account** for production workloads
3. **Grant roles at the narrowest scope** (resource > project > folder > org)
4. **Use predefined roles** instead of basic roles
5. **Create custom roles** if predefined roles grant too much
6. **Avoid service account keys** — use attached service accounts or Workload Identity instead
7. **Regularly audit** service account permissions using IAM Recommender

4. Assigning Service Accounts to Resources

Compute Engine VMs

```
# Assign a service account at VM creation
gcloud compute instances create my-vm \
  --zone=us-central1-a \
  --service-account=my-app-sa@PROJECT_ID.iam.gserviceaccount.com \
  --scopes=cloud-platform

# Change the service account on an existing VM (must stop first)
gcloud compute instances stop my-vm --zone=us-central1-a
gcloud compute instances set-service-account my-vm \
  --zone=us-central1-a \
  --service-account=my-app-sa@PROJECT_ID.iam.gserviceaccount.com \
  --scopes=cloud-platform
gcloud compute instances start my-vm --zone=us-central1-a

# Remove the service account from a VM
gcloud compute instances set-service-account my-vm \
  --zone=us-central1-a \
  --no-service-account --no-scopes
```

Scopes vs. IAM Roles

- **Scopes** are a legacy access control mechanism for VMs
- **IAM roles** are the modern, recommended way to control access
- Best practice: Use `--scopes=cloud-platform` (broadest scope) and control access through IAM roles
- The effective permissions are the **intersection** of scope AND IAM role

Effective Permission = Scope $\cap$ IAM Role

Cloud Run

```
# Assign a service account to a Cloud Run service
gcloud run deploy my-service \
  --image=IMAGE \
  --region=us-central1 \
  --service-account=my-app-sa@PROJECT_ID.iam.gserviceaccount.com
```

Cloud Functions

```
# Assign a service account to a Cloud Function
gcloud functions deploy my-function \
  --gen2 \
  --region=us-central1 \
  --runtime=python312 \
  --service-account=my-app-sa@PROJECT_ID.iam.gserviceaccount.com
```

GKE (Workload Identity)

```
# Annotate K8s service account with GCP service account
kubectl annotate serviceaccount my-ksa \
  --namespace=default \
  iam.gke.io/gcp-service-account=my-app-sa@PROJECT_ID.iam.gserviceaccount.com
```

5. Managing IAM of a Service Account

A service account is both a **principal** (can be granted roles) and a **resource** (has its own IAM policy controlling who can act on it).

Service Account IAM Roles

Role	Description
<code>roles/iam.serviceAccountUser</code>	Attach the SA to resources (VMs, Cloud Run, etc.)
<code>roles/iam.serviceAccountTokenCreator</code>	Create tokens for the SA (impersonation)
<code>roles/iam.serviceAccountAdmin</code>	Full management of the SA
<code>roles/iam.serviceAccountKeyAdmin</code>	Create and manage SA keys

Granting Permissions ON a Service Account

```
# Allow a user to use (attach) this service account to resources
gcloud iam service-accounts add-iam-policy-binding \
  my-app-sa@PROJECT_ID.iam.gserviceaccount.com \
  --member="user:alice@example.com" \
  --role="roles/iam.serviceAccountUser"

# Allow a user to create tokens (impersonate) this service account
gcloud iam service-accounts add-iam-policy-binding \
  my-app-sa@PROJECT_ID.iam.gserviceaccount.com \
  --member="user:admin@example.com" \
  --role="roles/iam.serviceAccountTokenCreator"

# View who has access to manage a service account
gcloud iam service-accounts get-iam-policy \
  my-app-sa@PROJECT_ID.iam.gserviceaccount.com

# Remove a binding
gcloud iam service-accounts remove-iam-policy-binding \
  my-app-sa@PROJECT_ID.iam.gserviceaccount.com \
  --member="user:alice@example.com" \
  --role="roles/iam.serviceAccountUser"
```

Important Distinction

- **Granting roles TO a SA** = What the SA can do (`gcloud projects add-iam-policy-binding`)
- **Granting roles ON a SA** = Who can use/manage the SA (`gcloud iam service-accounts add-iam-policy-binding`)

6. Managing Service Account Impersonation

What Is Service Account Impersonation?

- A user or service account can **act as** (impersonate) another service account
- Eliminates the need for **service account keys**
- Provides temporary, auditable access to a service account's permissions

How It Works

User Alice → Impersonates → SA-deploy → Accesses Cloud Storage

Requirements:

1. Alice needs roles/iam.serviceAccountTokenCreator on SA-deploy
2. SA-deploy needs roles/storage.objectAdmin on the bucket

Setting Up Impersonation

```
# Step 1: Grant Token Creator role on the target SA
gcloud iam service-accounts add-iam-policy-binding \
  target-sa@PROJECT_ID.iam.gserviceaccount.com \
  --member="user:alice@example.com" \
  --role="roles/iam.serviceAccountTokenCreator"

# Step 2: Use impersonation in gcloud commands
gcloud storage ls gs://my-bucket \
  --impersonate-service-account=target-sa@PROJECT_ID.iam.gserviceaccount.com

# Set impersonation globally for all gcloud commands
gcloud config set auth/impersonate_service_account target-sa@PROJECT_ID.iam.gserviceaccount.com

# Remove global impersonation
gcloud config unset auth/impersonate_service_account
```

Impersonation Chain

Service account A can impersonate B, and B can impersonate C (delegated):

```
gcloud storage ls gs://my-bucket \
  --impersonate-service-account=sa-c@PROJECT_ID.iam.gserviceaccount.com \
  --impersonation-delegation=sa-b@PROJECT_ID.iam.gserviceaccount.com
```

Benefits of Impersonation Over Keys

- **No key management** — No files to rotate, secure, or accidentally expose
- **Audit trail** — All impersonation is logged in audit logs
- **Time-limited** — Tokens expire automatically
- **Centralized control** — Manage access through IAM, not key distribution

7. Creating and Managing Short-Lived Service Account Credentials

Types of Short-Lived Credentials

Type	Duration	Use Case
Access token	1 hour (max)	API calls
ID token	1 hour (max)	Identity verification
Self-signed JWT	1 hour (max)	Google API access
Signed blob	N/A	Sign data
OIDC token	1 hour (max)	Authentication to Cloud Run, etc.

Generating Access Tokens

```
# Generate an access token for a service account
gcloud auth print-access-token \
  --impersonate-service-account=my-sa@PROJECT_ID.iam.gserviceaccount.com

# Use the token in API calls
TOKEN=$(gcloud auth print-access-token --impersonate-service-account=my-sa@PROJECT_ID.iam.gserviceaccount.com)
curl -H "Authorization: Bearer $TOKEN" \
  https://storage.googleapis.com/storage/v1/b/my-bucket/o
```

Generating ID Tokens

```
# Generate an ID token (for authenticating to Cloud Run, etc.)
gcloud auth print-identity-token \
  --impersonate-service-account=my-sa@PROJECT_ID.iam.gserviceaccount.com \
  --audiences=https://my-service-xxx-uc.a.run.app
```

Service Account Keys (Use Only When Necessary)

```
# Create a key (generates a JSON key file)
gcloud iam service-accounts keys create key.json \
  --iam-account=my-sa@PROJECT_ID.iam.gserviceaccount.com

# List keys for a service account
gcloud iam service-accounts keys list \
  --iam-account=my-sa@PROJECT_ID.iam.gserviceaccount.com

# Delete a key
gcloud iam service-accounts keys delete KEY_ID \
  --iam-account=my-sa@PROJECT_ID.iam.gserviceaccount.com
```

When to Use Keys vs. Alternatives

Method	When to Use	Security
Attached SA (metadata)	VMs, Cloud Run, Cloud Functions, GKE	Best (no keys needed)
Workload Identity	GKE pods	Best (no keys needed)
Impersonation	Human users needing SA permissions	Good (short-lived, auditable)
Workload Identity Federation	External workloads (AWS, Azure, GitHub)	Good (no keys needed)
SA keys	External systems that can't use federation	Worst (long-lived, must be rotated)

Workload Identity Federation

Allows external identities (AWS, Azure, GitHub Actions, etc.) to access Google Cloud without service account keys:

```
# Create a workload identity pool
gcloud iam workload-identity-pools create my-pool \
  --location=global \
  --description="My external identity pool"

# Create a provider (e.g., for GitHub Actions)
gcloud iam workload-identity-pools providers create-oidc github-provider \
  --location=global \
  --workload-identity-pool=my-pool \
  --issuer-uri="https://token.actions.githubusercontent.com" \
  --attribute-mapping="google.subject=assertion.sub,attribute.repository=assertion.repository"

# Allow the external identity to impersonate a service account
gcloud iam service-accounts add-iam-policy-binding \
  my-sa@PROJECT_ID.iam.gserviceaccount.com \
  --role="roles/iam.workloadIdentityUser" \
  --member="principalSet://iam.googleapis.com/projects/PROJECT_NUMBER/locations/global/workloadIdentityPools/my-pool/providers/github-provider"
```

8. Service Account Key Rotation

- Key rotation = generating a new key and deleting the old one
- Keys should be rotated regularly (best practice: every 90 days; org policy `iam.disableServiceAccountKeyCreation` prevents key creation entirely)

```
# Create a new key
gcloud iam service-accounts keys create new-key.json \
  --iam-account=MY_SA@PROJECT.iam.gserviceaccount.com

# List keys (note: only shows key IDs, not the private key itself)
gcloud iam service-accounts keys list \
  --iam-account=MY_SA@PROJECT.iam.gserviceaccount.com

# Delete old key by ID
gcloud iam service-accounts keys delete KEY_ID \
  --iam-account=MY_SA@PROJECT.iam.gserviceaccount.com

# Check key age
gcloud iam service-accounts keys list \
  --iam-account=MY_SA@PROJECT.iam.gserviceaccount.com \
  --format="table(name.basename(),validAfterTime,validBeforeTime)"
```

Org policy to disable key creation:

```
gcloud resource-manager org-policies enable-enforce \
  --organization=ORG_ID \
  constraints/iam.disableServiceAccountKeyCreation
```

Exam tip: Prefer Workload Identity Federation or service account impersonation over downloaded JSON keys. If keys must be used, rotate regularly and store in Secret Manager — never in code or environment variables.

9. Accessing Metadata Server

- GCP VMs, Cloud Run, Cloud Functions, and GKE pods can access credentials without a key file via the metadata server
- URL: <http://metadata.google.internal/computeMetadata/v1/> (only accessible from within GCP)

```
# Get access token (from inside a VM or container)
curl -H "Metadata-Flavor: Google" \
  "http://metadata.google.internal/computeMetadata/v1/instance/service-accounts/default/token"

# Get instance identity token (for authenticating to other services)
curl -H "Metadata-Flavor: Google" \
  "http://metadata.google.internal/computeMetadata/v1/instance/service-accounts/default/identity"

# Get service account email
curl -H "Metadata-Flavor: Google" \
  "http://metadata.google.internal/computeMetadata/v1/instance/service-accounts/default/email"
```

Application Default Credentials (ADC): Google client libraries automatically use the metadata server when running on GCP; no code changes needed.

```
# For local development: authenticate as your user
gcloud auth application-default login

# For local development: impersonate a service account
gcloud auth application-default login --impersonate-service-account=SA@PROJECT.iam.gserviceaccount.com
```

10. Cross-Project Service Account Usage

- A SA from Project A can be granted roles in Project B, and vice versa
- Use case: a Cloud Run service in project A needs to read from a Cloud Storage bucket in project B

```
# Grant SA from project A a role in project B
gcloud projects add-iam-policy-binding PROJECT_B \
  --member="serviceAccount:my-sa@PROJECT_A.iam.gserviceaccount.com" \
  --role="roles/storage.objectViewer"

# Grant SA from project A permission to be used by a resource in project B (SA user)
gcloud iam service-accounts add-iam-policy-binding my-sa@PROJECT_A.iam.gserviceaccount.com \
  --project=PROJECT_A \
  --member="serviceAccount:other-sa@PROJECT_B.iam.gserviceaccount.com" \
  --role="roles/iam.serviceAccountTokenCreator"
```

Exam tip: The `serviceAccountUser` role allows a principal to attach a SA to a resource (e.g., assign a SA to a VM). The `serviceAccountTokenCreator` role allows creating tokens on behalf of a SA (impersonation). These are different and commonly confused.

11. Service Account Best Practices Summary

Practice	Description
One SA per service	Don't share SAs across applications
Least privilege	Grant only needed permissions
Avoid keys	Use attached SAs, Workload Identity, or impersonation
Rotate keys	If keys are necessary, rotate them regularly
Monitor usage	Enable audit logs, use IAM Recommender
Disable unused SAs	Don't delete immediately — disable first
Don't use default SAs	Create dedicated SAs with specific roles
Use groups	Grant SA permissions through groups when possible
Short-lived credentials	Prefer tokens over long-lived keys
Restrict key creation	Use org policy <code>iam.disableServiceAccountKeyCreation</code>

Exam Practice Questions

1. A Compute Engine VM needs to read objects from Cloud Storage and publish messages to Pub/Sub. How should you configure access?

- Answer: Create a **dedicated service account** with `roles/storage.objectViewer` and `roles/pubsub.publisher`. Assign this SA to the VM with `--service-account` and `--scopes=cloud-platform`.

2. A developer needs to deploy Cloud Run services but should not be able to create VMs. They also need to attach a specific service account to Cloud Run. What roles do they need?

- Answer: `roles/run.developer` (deploy Cloud Run) and `roles/iam.serviceAccountUser` (on the specific service account they need to attach).

3. Your organization wants to prevent all service account key creation. How?

- Answer: Apply an **organization policy** with the constraint `constraints/iam.disableServiceAccountKeyCreation` set to enforce at the organization

level.

4. A GitHub Actions workflow needs to deploy to Google Cloud without storing service account keys as secrets. How?

- Answer: Use **Workload Identity Federation** — create a workload identity pool with a GitHub OIDC provider, then allow the pool to impersonate a Google Cloud service account.

5. What's the difference between `roles/iam.serviceAccountUser` and `roles/iam.serviceAccountTokenCreator` ?

- Answer: `serviceAccountUser` allows **attaching the SA to resources** (VMs, Cloud Run). `serviceAccountTokenCreator` allows **creating tokens as the SA** (impersonation). For deploying apps with an SA, you need User. For impersonation, you need TokenCreator.

6. A service account has `roles/editor` on the project but a VM using it has `--scopes=storage-read-only` . Can the VM write to Cloud Storage?

- Answer: **No**. Effective permissions are the intersection of scopes and IAM roles. The scope limits access to storage read-only, even though the IAM role would allow more. (This is why `--scopes=cloud-platform` is recommended.)

7. You need to temporarily grant a contractor the ability to act as a deployment service account. How do you do this securely?

- Answer: Grant `roles/iam.serviceAccountTokenCreator` on the deployment SA with a **conditional IAM binding** that includes a time-based expiration. The contractor uses `--impersonate-service-account` for their gcloud commands. No keys are created.

Glossary

Access token: A short-lived OAuth 2.0 credential (valid up to 1 hour) issued to a service account or user, used to authenticate requests to Google Cloud APIs via the `Authorization: Bearer` HTTP header.

ADC (Application Default Credentials): A Google client library mechanism that automatically discovers and uses credentials from the environment — metadata server on GCP, or `gcloud auth application-default login` locally — without requiring code changes.

App Engine: Google Cloud's fully managed platform-as-a-service; its default service account (`PROJECT_ID@appspot.gserviceaccount.com`) is assigned `roles/editor` by default, which is considered overly permissive.

Attached service account: A service account directly associated with a GCP resource (VM, Cloud Run service, Cloud Function, or GKE pod) so the resource can authenticate to Google APIs using the metadata server without needing a key file.

Audit log: A record in Cloud Logging of who performed which action on which resource; service account impersonation events are captured in audit logs, providing an audit trail.

AWS (Amazon Web Services): A public cloud provider whose workloads can authenticate to Google Cloud without service account keys using Workload Identity Federation with an AWS identity provider.

Azure: Microsoft's public cloud platform whose workloads can authenticate to Google Cloud without service account keys using Workload Identity Federation with an Azure identity provider.

CEL (Common Expression Language): The expression language used in conditional IAM bindings to restrict role grants by attributes such as request time; used when creating time-limited impersonation grants for contractors.

Cloud Function: Google Cloud's serverless, event-driven compute service; a service account can be assigned to a Cloud Function at deploy time using `--service-account`, replacing the default service account.

Cloud Run: Google Cloud's fully managed container execution environment; a service account is assigned at deploy time using `--service-account`, granting the service's identity for all outbound API calls.

Compute Engine: Google Cloud's virtual machine service; service accounts are attached to VMs at creation or later (after stopping) using `--service-account`, and access is governed by both IAM roles and OAuth scopes.

Default service account: A service account automatically created by Google when certain APIs are enabled; the Compute Engine default SA (`PROJECT_NUMBER-compute@developer.gserviceaccount.com`) and App Engine default SA are granted `roles/editor`, which is broader than recommended.

Delegated impersonation (impersonation chain): A pattern where service account A impersonates B, which in turn impersonates C, forming a delegation chain; used with `--impersonation-delegation` in gcloud.

GitHub Actions: A CI/CD platform that can authenticate to Google Cloud without service account keys using Workload Identity Federation with the GitHub OIDC provider (`token.actions.githubusercontent.com`).

GKE (Google Kubernetes Engine): Google Cloud's managed Kubernetes service; pods running in GKE clusters access Google Cloud APIs through Workload Identity, which maps a Kubernetes service account to a GCP service account.

Google-managed service account: A service account created and managed entirely by Google for internal GCP service agents (e.g., `service-PROJECT_NUMBER@compute-system.iam.gserviceaccount.com`); not directly managed by users.

IAM (Identity and Access Management): Google Cloud's access control system; service accounts are both principals (can receive role bindings) and resources (have their own IAM policy

controlling who can use or manage them).

IAM Recommender: A Google Cloud service that analyzes IAM usage over 90 days and suggests removing or reducing unused or over-provisioned service account role bindings.

ID token: A short-lived credential (valid up to 1 hour) issued as a signed JWT that identifies a service account; used to authenticate to services that verify Google-signed identity tokens, such as Cloud Run.

Impersonation: The act of a user or service account generating credentials on behalf of another service account, using the `roles/iam.serviceAccountTokenCreator` role; avoids the need for long-lived key files.

JSON key file: A downloadable file containing the private key of a service account, used to authenticate to Google Cloud from outside GCP; considered the least secure credential method because keys are long-lived and must be manually rotated and protected.

kubectl: The Kubernetes command-line tool; used to annotate Kubernetes service accounts with the `iam.gke.io/gcp-service-account` annotation to enable Workload Identity mapping.

Kubernetes service account (KSA): A Kubernetes-native identity assigned to pods within a GKE cluster; mapped to a Google Cloud service account via Workload Identity to grant GCP API access without key files.

Metadata server: A GCP-internal HTTP endpoint (`http://metadata.google.internal/computeMetadata/v1/`) accessible only from within GCP (VMs, Cloud Run, Cloud Functions, GKE pods) that provides instance metadata and short-lived access tokens for attached service accounts.

OIDC (OpenID Connect): An identity layer on top of OAuth 2.0 that issues signed JWT tokens for authentication; used by Workload Identity Federation providers (e.g., GitHub Actions, Azure) to establish trust with Google Cloud.

OIDC token: See ID token; a credential issued in the OIDC format used to authenticate to services that support OpenID Connect, such as Cloud Run.

Organization policy: A GCP governance control; the constraint `iam.disableServiceAccountKeyCreation` can be enforced at the organization level to prevent any service account key downloads across all projects.

Principal: In IAM, any identity that can be granted roles; service accounts are principals when they receive role bindings on resources, and resources when other principals are granted roles on them.

Pub/Sub: Google Cloud's managed messaging service; frequently used in service account examples where an SA is granted `roles/pubsub.subscriber` or `roles/pubsub.publisher` for specific topics.

roles/iam.serviceAccountAdmin: A predefined IAM role that grants full management of service accounts within a project, including creation, deletion, enabling, disabling, and key management.

roles/iam.serviceAccountKeyAdmin: A predefined IAM role that grants the ability to create, list, and delete service account keys without other service account management permissions.

roles/iam.serviceAccountTokenCreator: A predefined IAM role granted ON a service account that allows the grantee to generate access tokens, ID tokens, and self-signed JWTs on behalf of that service account (impersonation).

roles/iam.serviceAccountUser: A predefined IAM role granted ON a service account that allows the grantee to attach the service account to resources such as VMs or Cloud Run services.

roles/iam.workloadIdentityUser: A predefined IAM role granted ON a service account to allow an external identity (from a Workload Identity pool) to impersonate the service account.

roles/pubsub.publisher: A predefined IAM role that grants the ability to publish messages to a Pub/Sub topic.

roles/pubsub.subscriber: A predefined IAM role that grants the ability to consume messages from a Pub/Sub subscription.

roles/storage.objectViewer: A predefined IAM role that grants read-only access to Cloud Storage objects and their metadata.

Scope (OAuth scope): A legacy access control mechanism for Compute Engine VMs that limits which Google APIs the VM's service account can call; the effective permission is the intersection of the scope and the IAM role. Using `--scopes=cloud-platform` delegates full control to IAM roles.

Secret Manager: A Google Cloud service for storing and managing secrets such as API keys, passwords, and certificates; the recommended storage location for service account key files when keys cannot be avoided.

Self-signed JWT: A short-lived JSON Web Token signed with a service account's private key, used to authenticate directly to Google APIs that accept JWT-based authentication.

Service account: A special non-human Google account used by applications, VMs, and services to authenticate and authorize Google Cloud API calls; identified by the email format

`NAME@PROJECT_ID.iam.gserviceaccount.com` .

Service account disabling: The act of deactivating a service account so that all authentication attempts fail; IAM role bindings on the service account's resources remain intact and are restored immediately when the account is re-enabled.

Service account email: The unique identifier of a service account in the format

`SA_NAME@PROJECT_ID.iam.gserviceaccount.com` ; used as the member identifier in IAM policy bindings.

Service account impersonation: See Impersonation.

Service account key rotation: The process of creating a new service account key and deleting the old one to limit the exposure window of compromised credentials; best practice is rotation every 90 days or less.

Service account undelete: The ability to recover a deleted service account within 30 days of deletion by using its unique ID with `gcloud iam service-accounts undelete`.

Short-lived credential: Any time-limited authentication token (access token, ID token, OIDC token, or self-signed JWT) issued for a service account, with a maximum validity of 1 hour; preferred over long-lived JSON key files.

Signed blob: A cryptographic signature over arbitrary data produced using a service account's private key, via the `projects.serviceAccounts.signBlob` IAM API method.

Token Creator role: See `roles/iam.serviceAccountTokenCreator`.

Unique ID: A globally unique numeric identifier for a service account that persists even after the account's email is changed; used with `gcloud iam service-accounts undelete` to recover deleted accounts.

User-managed service account: A service account explicitly created by a developer or administrator (as opposed to a default or Google-managed account); the recommended type for production workloads because it carries only the roles explicitly granted to it.

Workload Identity: A GKE feature that binds a Kubernetes service account (KSA) to a Google Cloud service account (GSA) via an IAM annotation, allowing pods to access Google Cloud APIs without downloading a key file.

Workload Identity Federation: A GCP feature that allows workloads running outside of Google Cloud (on AWS, Azure, GitHub Actions, or any OIDC/SAML provider) to obtain short-lived Google Cloud credentials by exchanging their external identity token, eliminating the need for service account key files.

Workload Identity pool: A GCP resource that holds the configuration for trusting external identity providers in Workload Identity Federation; associated with one or more providers (OIDC, AWS, SAML).

Workload Identity pool provider: A configuration within a Workload Identity pool that defines how to trust tokens from a specific external identity system (e.g., GitHub Actions OIDC issuer, AWS STS), including attribute mappings used to authorize access.